

SOFTWARE ENGINEERING LAB – LAB 3

Component Modelling & Architectural Pattern Selection

Section H | Problem Statement #64: Disaster Relief Supply & Volunteer Coordinator

Student Name: Samvith Vaddiparthi SRN: PES1UG24AM468

Date: 10 September 2026

Architecture Selection Statement

"We chose Microservices Architecture for the Disaster Relief Supply & Volunteer Coordinator System."

1. Scenario Overview

The platform is an emergency disaster-management system that maps critical shelter supply needs (water, blankets, medical kits), matches requisitions against central warehouse stock and allocates delivery priority, dispatches volunteers by skill set, and tracks relief-vehicle deliveries for two actors: the Relief Volunteer and the Disaster Manager. Its key non-functional requirement (NFR-001) is that the field dashboard must operate on low network bandwidth with offline GIS data caching, and the system must remain operational while parts of the infrastructure fail.

2. Architectural Style Trade-off Analysis

Microservices Architecture – Selected. Decomposes the system into an API Gateway and independently deployable Supply Allocation, Volunteer Dispatch and Delivery Tracking services, each owning its own database. Bursty requisition matching and continuous GPS telemetry can scale independently, a failing service does not halt relief operations, and the field app keeps working offline.

Layered Architecture – Rejected. Organises the system into Presentation, Business Logic and Data layers inside one deployable. It offers clean separation of concerns, but the whole application must scale as one unit, so a telemetry-ingestion spike or a tracking fault would slow or block requisition and dispatch handling for every user.

Client-Server Architecture – Rejected. Centralises all logic on a single server with many field clients. Simple to deploy and consistent, but the server is a single point of failure and a scalability bottleneck – unacceptable in a disaster, where loss of the central server would halt requisitions, dispatch and tracking at once.

3. Technical Architectural Justification

Reason 1 – Independent scaling of unequal workloads. Immediately after a disaster, hundreds of shelters raise supply requisitions at once while every relief vehicle streams GPS telemetry continuously. These loads are very different in shape: requisition matching is bursty and compute-heavy, while telemetry ingestion is a constant high-volume stream. Microservices let the Supply Allocation Service and the Delivery Tracking Service scale out independently on their own worker nodes, so a surge in one never starves volunteer dispatch or the gateway. A Layered monolith would have to scale everything together, and a Client-Server design would funnel all of this through a single server bottleneck.

Reason 2 – Fault isolation in a degraded environment. Disaster environments have unreliable infrastructure, and the system must keep working while parts of it are failing. Because each service is isolated with its own data store, a crash or GPS-feed outage in the Delivery Tracking Service cannot take down requisition intake or volunteer dispatch, and the Notification Service can fail without blocking allocation decisions. The Field Mobile App keeps an offline GIS cache (NFR-001) and re-syncs through IReliefGateway when connectivity

returns, so field volunteers continue working through network loss – fault containment that a Client-Server architecture with a single central server cannot provide.

Security Advantage (NFR-002). The API Gateway is the only externally reachable component: it terminates TLS, authenticates every request and enforces role-based authorisation so that Relief Volunteers can submit requisitions and register skills while only Disaster Managers can allocate supplies, dispatch volunteers or reroute deliveries. All domain services sit on a private network behind the gateway with no public endpoints, and each owns its own database, so a compromise of one service (for example the Notification Service) cannot reach volunteer personal data in the Volunteer DB or alter warehouse stock in the Inventory DB. This limits the blast radius of any breach to a single service boundary.

Performance Benefit (NFR-001). The scenario's NFR demands low bandwidth consumption for field users. The Field Mobile App works from its offline GIS cache and only exchanges compact JSON deltas through the gateway, and vehicle positions are pushed to managers over a WebSocket from the Delivery Tracking Service instead of being polled. Internally, logistics manifests flow from Supply Allocation to Delivery Tracking over an asynchronous AMQP queue (IManifestIntake), so a volunteer submitting a requisition receives an immediate acknowledgement rather than waiting for manifest creation, and the telemetry-heavy tracking service can be scaled horizontally so that ETA and delay-escalation computations keep pace with the fleet during peak relief operations.

4. Components and Interfaces

Six components were identified from the scenario's functional requirements, actors and NFRs; each domain service owns its own database. Provided interfaces are drawn as balls and required interfaces as sockets in the diagram.

- **Field Mobile App (Client).** UI for Relief Volunteers and Disaster Managers with offline GIS cache and re-sync. Requires IReliefGateway.
- **API Gateway (Edge).** Authentication, role-based authorisation, routing and request aggregation. Provides IReliefGateway; requires ISupplyAllocation, IVolunteerDispatch, IDeliveryTracking.
- **Supply Allocation Service (Domain).** Requisition intake, severity scoring, warehouse matching and delivery-priority allocation; owns the Inventory DB. Provides ISupplyAllocation; requires IManifestIntake.
- **Volunteer Dispatch Service (Domain).** Volunteer skill registration, skill-to-need matching and dispatch; owns the Volunteer DB. Provides IVolunteerDispatch.
- **Delivery Tracking Service (Domain).** Vehicle status, location and ETA tracking, delay escalation and rerouting; owns the Delivery DB. Provides IDeliveryTracking and IManifestIntake; requires INotification; uses the external Relief Vehicle GPS Units.
- **Notification Service (Support).** SMS / push alerts to Disaster Managers and volunteers. Provides INotification.

5. UML 2.0 Component Diagram

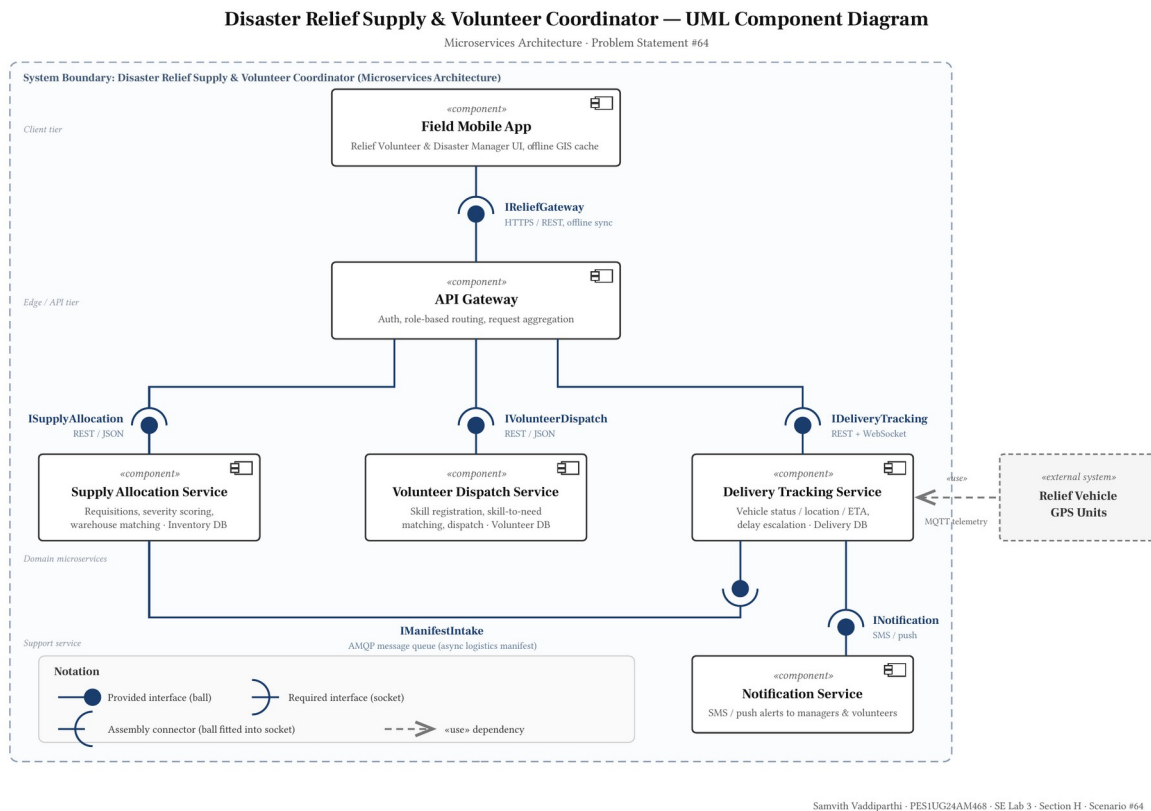


Figure 1. UML 2.0 Component Diagram – six components inside the system boundary, six ball-and-socket assembly connectors labelled with their protocols, and the «use» dependency on the external Relief Vehicle GPS Units.

6. Interface Interaction & Data Flow Walkthrough

- 1. Requisition intake.** A Relief Volunteer submits a shelter supply requisition from the Field Mobile App (or from its offline queue once connectivity returns). The app invokes IReliefGateway over «HTTPS / REST»; the API Gateway authenticates the volunteer and forwards the request through ISupplyAllocation via «REST / JSON».
- 2. Severity scoring and allocation.** The Supply Allocation Service computes the supply-need severity score, matches the requisition against Inventory DB stock and assigns delivery priority, giving water and medical needs the highest priority.
- 3. Manifest hand-over.** The allocated logistics manifest is published asynchronously to the Delivery Tracking Service over IManifestIntake via «AMQP», so the volunteer receives an immediate acknowledgement without waiting for delivery scheduling.
- 4. Volunteer dispatch.** A Disaster Manager reviews shelter needs and dispatches volunteers through IVolunteerDispatch via «REST / JSON»; the Volunteer Dispatch Service matches registered skill sets in the Volunteer DB to the need.
- 5. Delivery tracking and escalation.** Relief Vehicle GPS Units stream «MQTT» telemetry into the Delivery Tracking Service, which updates status, location and ETA in the Delivery DB and pushes them to managers over IDeliveryTracking via «REST + WebSocket». When a delivery is delayed, the service escalates it through INotification via «SMS / push» and the manager can reroute it.